SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1504

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**

Question Count: 5

Code of Conduct

I V.Raj kumar(192525208) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V.Raj kumar

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption,

and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

Solution:

1. Big Data Platform for Website, Mobile and Transaction Data

- **Distributed Storage:** Hadoop HDFS or cloud storage such as Amazon S3/Azure Data Lake is likely used to store billions of records. Data is distributed across multiple nodes for scalability and fault tolerance.
- **Data Ingestion:** Apache Kafka or Apache Flume can collect website clicks, mobile events, and transactions continuously.
- **Batch Processing:** Apache Spark or Hadoop MapReduce processes large historical datasets for reporting and trend analysis.
- **Data Organization:** Structured transaction data may be stored in relational databases or Hive tables, while semi-structured JSON/clickstream data is stored in data lakes. Hive partitions and indexes improve query performance.
- **Stream Processing:** Apache Spark Streaming, Apache Flink, or Kafka Streams can process events in real time for immediate analytics.
- **Scalability:** Data is partitioned by date, region, customer ID, or event type. Horizontal scaling allows additional nodes to handle increasing workloads.
- **Fault Tolerance:** Data replication and distributed processing ensure that node failures do not cause data loss.
- **Analytics and BI:** Processed data is exposed through Hive, Spark SQL, or cloud warehouses and connected to Tableau, Power BI, or similar BI tools for dashboards and business reporting.

Likely architecture:
Users → Kafka → HDFS/Data Lake → Spark/Hive → Analytics Layer → Power BI/Tableau

---

2. E-Commerce Personalization and Recommendation System

- **Event Streaming:** Apache Kafka is likely used to capture product views, cart additions, checkout events, and payment activity in real time.
- **Customer Profiles:** NoSQL databases such as MongoDB, Cassandra, or DynamoDB can maintain rapidly changing customer profiles and behavioral information.
- **Recommendation Workflow:** Events are collected, processed, transformed into customer features, and passed to machine-learning models for product recommendations.
- **Session Synchronization:** Kafka streams can synchronize clickstream, session, and transaction events. Customer IDs and session IDs connect browsing behavior with completed orders.
- **Caching:** Redis can store frequently accessed products, user profiles, recommendations, and session information to reduce database latency.
- **Query Processing:** Spark SQL, Trino, or Presto can perform distributed queries across large datasets.

- **Machine Learning:** Spark MLlib, Python-based ML frameworks, or cloud ML services can perform customer segmentation, recommendation, churn prediction, and purchase prediction.
- **Data Integration:** Structured order information from SQL databases can be combined with semi-structured browsing logs stored in a data lake.
- **Scalability and Fault Tolerance:** Kafka partitions distribute events across brokers, while database replication and Spark's distributed execution provide resilience.
- **Dashboards:** Processed KPIs such as conversion rate, abandoned carts, revenue, and customer activity can be displayed using Power BI, Tableau, or Grafana.

Likely architecture:
Customer Events → Kafka → Spark/Flink → Data Lake + Customer DB → ML Recommendation → Redis → Website

---

3. Healthcare Big Data System

- **Hybrid Database Architecture:** Structured patient records and prescriptions can be stored in relational databases such as PostgreSQL or MySQL. Diagnostic images and documents can be stored in object storage or distributed file systems.
- **Healthcare Data Integration:** Apache NiFi, Talend, or similar ETL tools can integrate hospital, laboratory, pharmacy, and insurance systems.
- **Interoperability:** HL7 and FHIR standards are likely used to exchange healthcare information between different systems.
- **Streaming:** Apache Kafka can ingest wearable sensor readings and monitoring data continuously.
- **Machine Learning:** Apache Spark, TensorFlow, or PyTorch can analyze patient history, laboratory results, images, and sensor data for disease-risk prediction.
- **Privacy:** Role-based access control ensures users only access information required for their role.
- **Encryption:** Data should be encrypted both during transmission and while stored.
- **Compliance:** Audit logs, authentication, authorization, data minimization, and controlled access support healthcare regulations such as HIPAA or applicable local regulations.
- **Clinical Analytics:** Patient data flows through cleaning, integration, feature extraction, analytics, and ML prediction before results are presented to clinicians.
- **Population Health:** Distributed processing enables analysis of large patient populations to identify disease patterns, risk factors, and treatment outcomes.

Likely architecture:
Hospitals/Labs/Wearables → FHIR/HL7 + Kafka → Data Lake/Databases → Spark/ML → Clinical Dashboard

---

4. Smart City Big Data Platform

- **Edge Computing:** Sensors and cameras can perform initial processing at the edge, reducing bandwidth and latency before sending important events to the central platform.
- **Messaging System:** Apache Kafka or MQTT can collect traffic, weather, transport, and CCTV metadata streams.
- **Central Storage:** HDFS, cloud object storage, or a data lake can store historical sensor and operational data.
- **Real-Time Processing:** Apache Flink, Spark Streaming, or Kafka Streams can calculate traffic congestion, vehicle density, and transport delays in real time.
- **Geospatial Analytics:** Location coordinates can be processed using PostGIS, GeoMesa, or spatial extensions to identify congestion zones and movement patterns.
- **Databases:** Time-series databases such as InfluxDB or TimescaleDB can store sensor measurements, while PostgreSQL/PostGIS can handle location-based queries.
- **Batch + Real-Time Analytics:** Historical data is processed using Spark for long-term urban planning, while streaming systems provide immediate traffic and transport insights.
- **Security:** Encryption, authentication, network segmentation, API security, and role-based access control protect infrastructure and citizen-related information.
- **Fault Tolerance:** Kafka replication, distributed storage, and redundant processing nodes help maintain availability.
- **Visualization:** Grafana, Power BI, Tableau, or custom dashboards can display live maps, traffic conditions, alerts, and infrastructure KPIs.

Likely architecture:
Sensors/CCTV → Edge → Kafka/MQTT → Flink/Spark → Time-Series DB + Data Lake → GIS Dashboard

---

5. Healthcare Analytics and Predictive Health System

- **Hybrid Storage:** Structured patient and appointment records can use PostgreSQL/MySQL, while medical reports, documents, and device data can be stored in a data lake or NoSQL database.
- **ETL Pipelines:** Apache NiFi, Talend, or Spark can extract, clean, transform, and integrate information from hospitals and clinics.
- **Interoperability:** HL7/FHIR APIs allow different healthcare systems to exchange patient information consistently.
- **Real-Time Monitoring:** Kafka can collect wearable-device readings, while Spark Streaming or Flink analyzes them for abnormal conditions.
- **Predictive Analytics:** Machine-learning models can use vital signs, laboratory values, medical history, and appointment patterns to calculate disease-risk scores.
- **Distributed Processing:** Apache Spark can process millions of patient records simultaneously for population-level research and statistical analysis.

- **Security:** Encryption at rest and in transit protects sensitive information. Authentication and role-based access control restrict unauthorized access.
- **Compliance and Auditing:** Access logs, audit trails, consent management, data retention policies, and regulatory controls support healthcare data governance.
- **Machine Learning:** Models can predict disease risk, identify high-risk patients, support diagnosis, and recommend possible treatment options for clinician review.
- **Analytics Dashboard:** Doctors and administrators can view patient risk scores, population trends, treatment outcomes, and real-time alerts through secure dashboards.

Likely architecture:
EHR + Labs + Wearables → ETL/FHIR → Data Lake + SQL DB → Spark → ML Models → Risk Scores → Clinical Dashboard